

Day 1 – Messy data, profiling, and learning to distrust a file

Time budget: 5 to 7 hours **Theme:** You cannot fix what you have not measured.

Learning objectives

By the end of today you should be able to answer these out loud, without notes.

1. What is data profiling and why does it happen *before* cleaning?
 2. What is referential integrity and how do you test it with a join?
 3. Why does whitespace on a join key cost more than a null does?
 4. What is the difference between a column being *null* and being *wrong*?
 5. Why is a fault that is easy to see less dangerous than one that is not?
-

What you have been given

Run this if you have not already.

```
cd "C:\Monash University\Projects\Tax Project"
python scripts/generate_data.py
```

That writes:

data/raw/erp_sap_au_transactions.csv	18,182 rows	SAP field names
data/raw/erp_dynamics_uk_transactions.csv	14,061 rows	Dynamics field names
data/raw/erp_oracle_us_transactions.csv	12,076 rows	Oracle field names
data/raw/erp_netsuite_nz_transactions.csv	5,981 rows	NetSuite field names
data/raw/customers.csv	4,000 rows	
data/raw/vendors.csv	800 rows	
data/reference/*.csv		jurisdictions, codes, rates, categories
data/golden/golden_dataset.csv	250 rows	verified expectations
data/_answer_key/	DO NOT OPEN YET	

The answer key is real. data/_answer_key/fault_log.csv lists every fault that was deliberately injected, by transaction id. You will use it at the end of Day 1 to score yourself. Opening it before you have written your profiler is cheating yourself out of the exercise, and the whole point of this project is that you can measure your own recall.

12,468 faults were injected into 50,300 delivered rows. Some are loud. Some are designed to be invisible on a summary and only findable by reconciliation.

Concept 1 – Why profile first

DELOITTE RELEVANCE. On a real engagement you get a client extract on a Friday and you are expected to say something intelligent about it on Monday. The first thing you produce is a profile, not a fix. A profile tells you whether the file is even usable, gives you the questions to ask the client, and becomes the baseline you compare next month's extract against.

A profile answers, for every column:

- How much of it is missing?
- How many distinct values are there, and is that plausible?
- What are the extremes?
- What type is it *really*, as opposed to what it claims to be?
- Do the values that should point at another table actually resolve?

PRODUCTION ALERT. The most expensive profiling finding is usually *cardinality drift*: last month currency_code had 4 distinct values, this month it has 11. Nobody told you. A source system started sending A\$.

Concept 2 – Loud, medium and subtle faults

Tier	Example in this dataset	Found by
LOUD	customer_id is null, date reads TBC	A profile
MEDIUM	Tax code ZZ-UNKNOWN not in the reference table	A join
SUBTLE	Tax understated by 0.8 per cent	Reconciliation only

Today you build the tool that catches the LOUD tier and most of the MEDIUM tier. The SUBTLE tier is Day 4, and it is where the real money is.

INTERVIEW ALERT. "How would you approach a dataset you have never seen?" The structured answer is: profile, then check referential integrity, then check business rules, then reconcile against an independent source. In that order, because each stage assumes the previous one passed.

Concept 3 – The whitespace problem

1,497 rows in this dataset have a customer_id like " CUST001186 ".

A null customer id is *loud*. Your profile shows 600 nulls and you go and ask.

A whitespace-padded customer id is *silent*. It is not null. It looks correct in Excel. It joins to nothing, so the customer's exemption status is never found, so the transaction is taxed at the standard rate, so your filing is wrong and nothing anywhere reported an error.

That single idea is most of what separates a data analyst from a data engineer.

Your first exercise

Build the profiling framework. This is yours to write, not mine.

Task

Create `src/taxtrace/profiling/profiler.py` with one public function:

```
def profile_dataframe(df: pd.DataFrame, table_name: str) → pd.DataFrame:
    """Return one row per column describing that column."""
```

The returned frame must have one row per column of `df` and these columns:

Output column	Meaning
<code>table_name</code>	The name you passed in
<code>column_name</code>	Name of the column being profiled
<code>dtype</code>	Pandas dtype as a string
<code>row_count</code>	Total rows in the table
<code>null_count</code>	Nulls, including empty strings and whitespace-only strings
<code>null_pct</code>	Null count as a percentage, rounded to 2dp
<code>distinct_count</code>	Number of distinct non-null values
<code>distinct_pct</code>	Distinct as a percentage of non-null, rounded to 2dp
<code>min_value</code>	Minimum, as a string. Blank if not sensible
<code>max_value</code>	Maximum, as a string. Blank if not sensible
<code>mean_value</code>	Mean for numeric columns only, else blank
<code>top_value</code>	Most common non-null value
<code>top_value_pct</code>	Share of non-null rows holding that value, 2dp
<code>has_leading_trailing_spaces</code>	True if any value has surrounding whitespace
<code>sample_values</code>	Three example non-null values, pipe separated

Rules

1. It must work on a column of **any** dtype without raising. Every column in

these files is read as text by default, so do not assume numbers.

1. Treat "", " " and "N/A" as null for `null_count`. Real profilers have

a configurable null-token list. Make yours a parameter with a default.

1. Do not mutate the input frame. Profiling that changes the thing it profiles

is a bug you will only find at 2am.

1. No hardcoded column names anywhere.

Then run it

Write `scripts/run_profile.py` that profiles all four ERP extracts plus `customers.csv`, concatenates the results, and writes `reports/profiling/data_profile.csv`.

Read every file with `dtype=str` so pandas does not silently coerce anything. That is deliberate. You want to see the raw text.

Questions to answer in writing when you are done

Put your answers in `docs/day01_findings.md`. Three or four sentences each.

1. Which column across all four files has the highest null percentage, and what would you ask the client about it?

1. `LAND1` in the SAP file and `CountryRegionCode` in the Dynamics file both hold country. How many distinct values does each have? What does that tell you about standardising them?

1. Find one column where `distinct_count` equals `row_count`. What does that tell you about the column, and is it true in every file?

1. Pick the two faults you think are most dangerous and say why. Not the most common. The most dangerous.

Stretch, only if you have time

Add a `-html` flag that also writes `reports/profiling/data_quality_report.html`. Plain HTML from `DataFrame.to_html` is fine. Do not install anything.

Checkpoint

You are done with Day 1 when:

- ☐ `profile_dataframe` runs on all five files without raising
- ☐ `reports/profiling/data_profile.csv` exists and has one row per column per table
- ☐ You have written `docs/day01_findings.md`
- ☐ You can explain, out loud, why whitespace on a key is worse than a null
- ☐ You have committed your work on a branch called `feature/day1-profiling`

Do not open the answer key until the checkpoint is complete. When it is, tell me and I will review your implementation line by line, show you what a production version looks like, and only then we will score your profiler against the fault log together.

Hints, if you get stuck

Read these one at a time, not all at once.

Hint 1, structure

Loop over `df.columns`, build a dict per column, then `pd.DataFrame(rows)` at the end. Do not try to do it with a single clever aggregation.

Hint 2, the null definition

`s.isna()` catches real nulls. For the token nulls you need something like `s.astype(str).str.strip().isin(null_tokens)`. Combine with `|`.

Hint 3, min and max on text columns

Try `pd.to_numeric(s, errors="coerce")`. If most values convert, treat the column as numeric for min, max and mean. If not, fall back to string min and max, which is still useful for spotting a stray value.

Hint 4, whitespace detection

`(s.astype(str) != s.astype(str).str.strip()).any()`